

1.1.1 Les méthodes Agile pour le développement logiciel

EN BREF

Hublot est développé selon une démarche **Agile pragmatique** : chaque évolution part d'un **besoin métier RH** (user story), se valide par des **critères d'acceptation testables**, et n'est livrée qu'une fois la **Definition of Done** remplie (CI verte, tests documentés). Ce chapitre explique ce cadre, ce qui existe dans le dépôt, et comment il se relie à la chaîne DevOps (document 1.1.2).

Objectif pédagogique

Comprendre et appliquer les **méthodes Agile** pour structurer le travail logiciel : backlog, user stories, critères d'acceptation, Definition of Done, et boucle de feedback rapide.

Contexte — projet Hublot

Hublot est un **tableau de bord RH** pour la DIRM Méditerranée : effectifs, missions, régions, services, parité, etc. Les utilisateurs sont des agents et responsables qui **pilotent des effectifs** ; une erreur de chiffre ou un accès non contrôlé a un impact métier direct. Le développement ne peut donc pas se limiter à « coder des écrans » : chaque livraison doit être **traçable, testée et alignée sur un besoin exprimé**.

Définitions

Terme	Définition
Agile	Approche itérative qui livre de la valeur par petits incréments, avec retour rapide plutôt qu'un cahier des charges figé.
Backlog	Liste priorisée des besoins à traiter (fonctionnalités, corrections, dette technique).
User story	Formulation d'un besoin : En tant que [rôle], je veux [action], afin de [bénéfice].
Critères d'acceptation	Conditions observables de succès, souvent au format Étant donné / Quand / Alors (Given / When / Then).
Definition of Done (DoD)	Liste de contrôles obligatoires avant de considérer une évolution comme terminée et livrable.
Incrément	Version fonctionnelle du produit après une itération (merge sur <code>main</code> + déploiement).

Ce qui a été mis en place

Nous n'avons pas mis en place un framework Scrum complet (pas de sprint calendarisé ni de rôles PO/SM formalisés), mais **les artefacts Agile essentiels** sont réels et utilisables :

1. **Modèle de user story** dans GitHub (`.github/ISSUE_TEMPLATE/user-story.yml`) : contexte, story, critères Given/When/Then, checklist DoD.
2. **Scénarios de test ST-F*** et **ST-SEC*** dans le document 1.2.2 : chaque scénario relie un besoin à une preuve d'exécution.
3. **Definition of Done** opérationnelle dans les documents 1.2.5 et 1.2.6 : CI verte, statuts Passé/Échec, rapport daté.
4. **Boucle de feedback automatique** : à chaque push, le workflow **CI** exécute lint, 48 tests unitaires, audit npm, build et E2E — une régression est détectée en quelques minutes.
5. **Priorisation par risque métier** : sécurité d'accès, fiabilité des données RH, lisibilité des tableaux de bord.

Pourquoi ces choix

Choix	Justification
Agile pragmatique (pas Scrum strict)	Équipe réduite, projet Studi + usage interne DIRM : l'important est la traçabilité et les tests , pas la cérémonialité.
Critères d'acceptation testables	Évite le flou (« ça marche ») ; aligne développement et validation jury.
DoD incluant la CI	Impossible de merger une régression silencieuse si les checks sont requis avant déploiement Netlify.
Scénarios ST-* numérotés	Permet de prouver au jury quel besoin est couvert et comment il a été validé.

Comment ça fonctionne

Du besoin à la livraison

- 1 **Expression du besoin** — Exemple réel : « Les responsables veulent filtrer par **unité** à l'intérieur d'un service. »
- 2 **User story** — En tant que responsable RH, je veux filtrer par unité lorsque j'ai sélectionné un service, afin d'analyser un sous-ensemble précis d'effectifs.
- 3 **Critères d'acceptation** — Étant donné un service sélectionné, quand j'ouvre le filtre unité, alors seules les unités de ce service apparaissent ; quand je change de service, le filtre unité se réinitialise.
- 4 **Implémentation** — Code + tests unitaires si logique métier ; mise à jour du scénario ST-F03 si parcours UI.
- 5 **Vérification DoD** — `npm run lint`, `npm run test:run`, `npm run audit:prod`, `npm run build`, `npm run test:e2e` (ou CI distante).
- 6 **Merge et déploiement** — Push sur `main` → CI verte → Netlify déploie (voir 1.1.5).
- 7 **Capitalisation** — Statut **Passé** dans le plan de test (1.3.7) et, si campagne manuelle, entrée dans le rapport d'exécution (1.2.9).

Artefacts et emplacements

Artefact	Où dans le projet	Usage concret
User story	Issue GitHub (template)	Créer une demande structurée avant de coder
Critères d'acceptation	1.2.2 Elaborer un scénario de test	Lier besoin test
DoD	1.2.5 / 1.2.6	Checklist avant merge
Feedback CI	<code>.github/workflows/ci.yml</code>	Qualité automatique à chaque push
Rapport	1.2.9 Rapport d'exécution des tests	Preuve datée pour le jury

Quand

Moment	Action Agile
Avant de coder	Rédiger ou mettre à jour la user story et les critères ST-*
Pendant le dev	Commits fréquents, tests locaux
À chaque push / PR	CI exécutée (lint, tests, audit, build, E2E)
Avant merge sur <code>main</code>	DoD complète (CI verte + doc à jour)
Après campagne de tests	Mise à jour des statuts Passé/Échec (1.2.6)

Preuves et démonstration

Créer une user story (GitHub) : dépôt → Issues → New issue → template User Story.

Vérifier la DoD localement avant merge :

TERMINAL

```
npm run lint
npm run test:run # 48 tests Vitest
npm run audit:prod
npm run build
npm run test:e2e
```

Check-list opérationnelle (avant merge) :

- Besoin reformulé en user story claire
- Critères d'acceptation ajoutés ou mis à jour (scénarios ST-*)
- CI verte sur la branche

- Statuts mis à jour dans le plan de test (1.3.7)
 - Rapport ou preuve d'exécution disponible si campagne manuelle
-

SYNTHÈSE

« Sur Hublot, l'Agile n'est pas théorique : chaque évolution est formulée en **user story** avec **critères d'acceptation** testables (ST-F* / ST-SEC*), et rien n'est considéré terminé sans **Definition of Done** — dont une **CI verte** à chaque intégration. Cela aligne les livraisons fréquentes (DevOps) sur la **valeur métier RH** et la **qualité mesurable**. »

Documents liés

- › 1.1.2 La démarche DevOps — chaîne Dev → CI → CD
- › 1.2.2 Elaborer un scénario de test — scénarios ST-*
- › 1.2.5 Planifier efficacement les tests — pyramide et DoD
- › 1.2.6 Valider les résultats des tests — Passé / Échec